

প্রতিটি Section-এর পেছনের তত্ত্ব — সূত্রসহ

কোড ব্যাখ্যার সঙ্গী ডকুমেন্ট · কোন section-এ কোন theory কাজ করছে, গণিত কী বলছে, কেন এভাবে কাজ করে
Term ও সূত্র ইংরেজিতে, ব্যাখ্যা বাংলায়

SECTION 3 — Data Cleaning-এর পেছনের তত্ত্ব

Missing Data কেন সমস্যা

পরিসংখ্যানে missing data তিন ধরনের হয়:

ধরন	মানে	উদাহরণ
MCAR (Missing Completely At Random)	ফাঁকা হওয়াটা সম্পূর্ণ এলোমেলো, কোনো কারণ নেই	যন্ত্র হঠাৎ নষ্ট হয়ে একটা মাপ রেকর্ড হয়নি
MAR (Missing At Random)	অন্য কোনো column-এর উপর নির্ভর করে ফাঁকা	নারীরা বয়স বলতে চাননি → age ফাঁকা, কারণ sex
MNAR (Missing Not At Random)	মানটা নিজেই কারণ	বেশি আয়ের লোক আয় গোপন করেছেন

Median imputation শুধু MCAR ও MAR-এর ক্ষেত্রে মোটামুটি নিরাপদ। MNAR হলে যেকোনো imputation পক্ষপাত (bias) আনে — তখন সেই তথ্যটা "ফাঁকা ছিল" এই বিষয়টাই আলাদা feature হিসেবে রাখা উচিত।

কেন median, কেন mean —গাণিতিক কারণ:

Mean = $\sum x_i / n$ → এখানে প্রতিটি মান যোগ হয়, তাই একটা চরম মান পুরো যোগফলকে টেনে নেয়। **Breakdown point = 0%**।

Median = মাঝের মান → শুধু ক্রম দেখে, মান দেখে না। **Breakdown point = 50%**, অর্থাৎ অর্ধেক ডেটা নষ্ট হলেও median টলে না।

Duplicate Row কেন বাদ — তত্ত্ব

Machine learning-এর মূল অনুমান হলো observation গুলো **i.i.d.** (independent and identically distributed) — প্রতিটি sample স্বাধীন। Duplicate থাকলে স্বাধীনতা ভেঙে যায়। আরও বড় সমস্যা: train ও test একে অপর থেকে স্বাধীন হওয়ার শর্ত ভাঙে, ফলে test আর সত্যিকারের "অদেখা ডেটা" থাকে না। একে বলে **optimistic bias** — measured accuracy সত্যিকারের generalization-এর চেয়ে বেশি দেখায়।

Label Encoding-এর তত্ত্ব ও সীমাবদ্ধতা

LabelEncoder প্রতিটি category-কে একটা পূর্ণসংখ্যা দেয়: red→0, green→1, blue→2।

সমস্যা — কৃত্রিম ক্রম (artificial ordinality): সংখ্যা দিলেই model ভাবে blue(2) > green(1) > red(0), আর "blue - red = 2" মানে কিছু একটা। কিন্তু রঙের তো কোনো ক্রম নেই!

কোন model-এ ক্ষতি বেশি: Logistic Regression, SVM, KNN — এরা সংখ্যার মান ও দূরত্ব নিয়ে হিসাব করে, তাই ভুল ক্রম ক্ষতি করে।

কোন model-এ ক্ষতি কম: Decision Tree, Random Forest — এরা শুধু "> না <" দেখে ভাগ করে, তাই কম ক্ষতিগ্রস্ত।

সঠিক বিকল্প: `pd.get_dummies()` বা One-Hot Encoding — প্রতিটি category-র জন্য আলাদা 0/1 column। তবে category বেশি হলে column বিস্তারিত ঘটে, তাই lab-এ LabelEncoder-ই চলে।

SECTION 6 — Train/Test Split ও Scaling-এর তত্ত্ব

Generalization — কেন ভাগ করি

আমরা যা কমাতে চাই তা হলো **true error** (সব সম্ভাব্য ডেটায় ভুল), কিন্তু মাপতে পারি শুধু **empirical error** (হাতের ডেটায় ভুল)। Training data-তে মাপলে model সেটা মুখস্থ করেই কম error দেখাতে পারে। তাই আলাদা test set লাগে — এটাই **generalization** মাপার একমাত্র সং উপায়।

Bias-Variance Trade-off (এটা theory exam-এও আসে)

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Var}[\hat{f}(x)] + \sigma^2$$

(underfit) (overfit) (irreducible error)

- **Bias** — জটিল বাস্তবতাকে সরল model দিয়ে ধরার ভুল। বেশি হলে **underfitting**: train ও test দুটোতেই খারাপ।
- **Variance** — ভিন্ন training data দিলে model কতটা বদলে যায়। বেশি হলে **overfitting**: train-এ চমৎকার, test-এ খারাপ।
- σ^2 (**irreducible error**) — ডেটার নিজস্ব এলোমেলোভাব। যত ভালো model-ই বানাও, **এটা কখনো দূর হয় না**।

Model-এর flexibility বাড়ালে bias কমে কিন্তু variance বাড়ে। Test error তাই **U-আকৃতির** — মাঝখানে সবচেয়ে ভালো জায়গা।

Feature Scaling-এর গণিত

Standardization (Z-score): $z = (x - \mu) / \sigma \rightarrow$ ফলাফলে mean = 0, std = 1

Normalization (Min-Max): $x' = (x - \min) / (\max - \min) \rightarrow$ ফলাফল 0 থেকে 1

কেন লাগে — মূল কারণ Euclidean distance-এর সূত্র:

$$d(p, q) = \sqrt{[(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2]}$$

এখানে প্রতিটি পদ **বর্গ** হয়। তাই বড় unit-এর feature-এর পার্থক্য বর্গ হয়ে বিশাল হয়ে যায়, আর ছোট unit-এর feature কার্যত অদৃশ্য হয়ে যায়। উদাহরণ: fare-এ ১০০ পার্থক্য $\rightarrow$ ১০,০০০; sex-এ ১ পার্থক্য $\rightarrow$ ১। অর্থাৎ দূরত্বের ৯৯.৯৯% আসে শুধু fare থেকে।

Scaling লাগে	কারণ	Scaling লাগে না	কারণ
KNN, SVM, K-Means	দূরত্ব হিসাব করে	Decision Tree	শুধু threshold-এ ভাগ করে, দূরত্ব মাপে না
PCA	variance দেখে, বড় unit-এর variance এমনিতেই বড়	Random Forest	tree-র সমষ্টি, একই কারণ
Logistic/Linear Regression	gradient descent দ্রুত converge করে	Naive Bayes	প্রতিটি feature আলাদাভাবে distribution ধরে

Data Leakage — তাত্ত্বিক সংজ্ঞা

Data leakage হলো এমন পরিস্থিতি যেখানে training-এর সময় model এমন তথ্য পেয়ে যায় যা prediction-এর সময় বাস্তবে পাওয়া যেত না।
তিন রকম:

1. **Target leakage** — কোনো feature আসলে উত্তরটাই বলে দেয় (titanic-এর `alive`)।
2. **Train-test contamination** — test set-এর তথ্য preprocessing-এ ঢুকে যায় (পুরো ডেটায় scaler fit করা)।
3. **Duplicate leakage** — একই row train ও test দুটোতেই।

এজন্যই `fit_transform` শুধু train-এ, `transform` test-এ। μ ও σ হলো **শেখা parameter** — এগুলো training data থেকেই আসতে হবে।

SECTION 7 — PCA-র সম্পূর্ণ গণিত

লক্ষ্য

PCA খোঁজে feature গুলোর এমন **normalized linear combination** যার variance সর্বোচ্চ:

$$Z_1 = \phi_{11}X_1 + \phi_{21}X_2 + \dots + \phi_{p1}X_p \quad \text{শর্ত: } \sum_j \phi_{j1}^2 = 1$$

শর্তটা কেন? শর্ত না থাকলে ϕ -গুলো যত খুশি বড় করে variance অসীম করা যেত — সমস্যাটার কোনো সমাধানই থাকত না। ϕ -গুলোকে বলে **loadings**।

Eigenvalue-Eigenvector দিয়ে সমাধান

Centered ডেটার covariance matrix C বের করে সমাধান করি:

$$C \mathbf{v} = \lambda \mathbf{v} \quad \text{অর্থাৎ } \det(C - \lambda I) = 0$$

এখানে $\mathbf{v}$ = eigenvector = principal component-এর দিক

λ = eigenvalue = ওই দিকে কত variance আছে

eigenvector কেন? Eigenvector হলো সেই বিশেষ দিক যদিকে matrix শুধু টানে বা ছোট করে, **দিক বদলায় না**। Covariance matrix-এর ক্ষেত্রে সবচেয়ে বড় λ -এর দিকটাই ডেটার সবচেয়ে ছড়ানো দিক।

$$\text{Proportion of Variance Explained: } PVE_k = \lambda_k / (\lambda_1 + \lambda_2 + \dots + \lambda_p)$$

কোডে `pca.explained_variance_ratio_` ঠিক এটাই দেয়। যোগফল নিলে মোট কত variance রাখা হলো।

দ্বিতীয় component ও Orthogonality

Z_2 হলো সেই combination যার variance সর্বোচ্চ, **কিন্তু Z_1 -এর সাথে uncorrelated**। গাণিতিকভাবে " Z_2 uncorrelated with Z_1 " আর " $\phi_2 \perp \phi_1$ (লম্ব)" — একই কথা। তাই PC গুলো পরস্পর লম্ব একটা নতুন coordinate system বানায়।

PCA-র মূল অনুমান (assumptions) — viva-তে জিজ্ঞেস করতে পারে:

1. **Linearity** — সম্পর্কগুলো linear ধরে নেয়। বাঁকা (non-linear) structure ধরতে পারে না; সেখানে t-SNE/UMAP লাগে।

2. বেশি **variance = বেশি তথ্য** — এটা সবসময় সত্যি নয়! কম variance-এর কোনো feature-ও class আলাদা করতে গুরুত্বপূর্ণ হতে পারে।
3. **Component গুলো orthogonal** — বাস্তব কারণগুলো লম্ব নাও হতে পারে।
4. **Scaling করা আছে** — না করলে বড় unit-এর feature সব দখল করে নেবে।

PCA unsupervised — এটাই তার শক্তি ও দুর্বলতা। সে y দেখে না, শুধু X-এর ছড়ানো দেখে। তাই যদি class আলাদা করার তথ্যটা কম-variance দিকে থাকে, PCA সেটাই ফেলে দেবে। তখন accuracy কমবে। **LDA** এই সমস্যার সমাধান — সে supervised, class আলাদা করার দিক খোঁজে (সর্বোচ্চ C-1 component)।

SECTION 8 — প্রতিটি Supervised Algorithm-এর তত্ত্ব

১. Logistic Regression

$$p = 1 / (1 + e^{-z}) \quad \text{যেখানে } z = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p \quad (\text{sigmoid})$$

$$\text{odds} = p/(1-p) \quad \cdot \quad \log(\text{odds}) = \text{logit} = \beta_0 + \beta_1 x$$

log-odds কেন? Probability 0 থেকে 1-এর মধ্যে আটকা, linear সমীকরণ দিয়ে সরাসরি model করা যায় না। Odds হয় 0 থেকে ∞ (অসমান)। কিন্তু **log নিলে $-\infty$ থেকে $+\infty$** হয় এবং 0-এর চারপাশে প্রতিসম হয় — তখন linear model বসানো যায়।

Fit হয় কীভাবে? Least squares নয়, **Maximum Likelihood Estimation (MLE)** দিয়ে। Cost function:

$$\text{Log-loss} = -(1/n) \sum [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

এর closed-form সমাধান নেই, তাই **gradient descent** লাগে — এজন্যই `max_iter=1000` দিতে হয়।

২. KNN (K-Nearest Neighbours)

কোনো training নেই — একে বলে **lazy learner** বা **instance-based learning**। পুরো training data মনে রাখে, আর predict-এর সময় নিকটতম K জনের ভোটা নেয়।

K ছোট (K=1)	K বড় (K=50)
Low bias, high variance → overfit	High bias , low variance → underfit
boundary খুব এবড়োখেবড়ো	boundary খুব মসৃণ

Curse of Dimensionality — KNN-এর প্রধান শত্রু। Dimension বাড়লে সব point প্রায় **সমান দূরে** হয়ে যায়, তখন "নিকটতম প্রতিবেশী" ধারণাটাই অর্থহীন। এজন্যই PCA করে dimension কমালে KNN-এর ফল প্রায়ই ভালো হয়।

৩. SVM (Support Vector Machine)

দুই class-এর মাঝে এমন hyperplane খোঁজে যার **margin সর্বোচ্চ**:

$$\text{Margin} = 2/\|w\| \rightarrow \text{maximize করা মানে minimize } \frac{1}{2}\|w\|^2 \quad (\text{শর্ত: } y_i(w \cdot x_i + b) \geq 1)$$

Support vector = margin-এর গায়ে বা ভেতরে থাকা point গুলো। **শুধু এরাই boundary ঠিক করে** — বাকি point সরিয়ে দিলেও কিছু বদলায় না। এটাই SVM-এর সবচেয়ে সুন্দর বৈশিষ্ট্য।

Soft margin (C parameter): বাস্তবে ডেটা পুরোপুরি আলাদা হয় না, তাই কিছু ভুল মেনে নেওয়া হয়। C বড় = ভুল কম মানবে (overfit ঝুঁকি), C ছোট = margin চওড়া, বেশি সহনশীল।

Kernel trick: $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$
RBF kernel: $K(a, b) = \exp(-\gamma \|a - b\|^2)$

Kernel-এর জাদু: ডেটাকে সত্যিই উঁচু dimension-এ **না নিয়েই**, যেন নেওয়া হয়েছে সেভাবে dot product হিসাব করে। ফলে বিশাল computation বেঁচে যায় অথচ বাঁকা boundary আঁকা যায়।

8. Decision Tree

Gini Impurity = $1 - \sum (p_i)^2$ · **Entropy** = $-\sum p_i \log_2(p_i)$
Information Gain = parent impurity - $\sum (n_i/n) \times \text{child impurity}$

প্রতিটি node-এ সব সম্ভাব্য split পরীক্ষা করে, যেটা impurity সবচেয়ে বেশি কমায় সেটাই বেছে নেয়। উদাহরণ: ৩টা হ্যাঁ, ১টা না → $Gini = 1 - (0.75^2 + 0.25^2) = 1 - 0.625 = 0.375$ । $Gini = 0$ মানে **pure** (সব একই class)।

Gini না Entropy? দুটোর ফল প্রায় একই। Gini দ্রুত (log হিসাব লাগে না), তাই sklearn-এর default। Entropy তথ্যতত্ত্ব (information theory) থেকে এসেছে।

Regression Tree-এ Gini-র বদলে **SSR** (sum of squared residuals) কমানো হয়, আর leaf-এ prediction = ওই leaf-এর y-গুলোর গড়।

Regression Tree extrapolate করতে পারে না — training range-এর বাইরে গেলে সে শুধু নিকটতম leaf-এর ধ্রুবক মান দেয়, তাই prediction সমতল হয়ে যায়। Linear Regression পারে।

৯. Random Forest

দুইটা randomness একসাথে:

- Bootstrap (Bagging)** — n টা row **ফেরত দিয়ে** (with replacement) তুলে প্রতিটি tree-র জন্য আলাদা dataset বানায়।
- Random feature subset** — প্রতিটি node-এ সব feature নয়, শুধু $\sqrt{p}$ টা random feature বিবেচনা করে।

দ্বিতীয়টা কেন অপরিহার্য? শুধু bootstrap দিলে প্রতিটি tree সবচেয়ে শক্তিশালী feature-টাকেই প্রথমে বাছত, ফলে সব tree প্রায় একরকম হতো — গড় নিয়ে কোনো লাভ হতো না। Random feature subset tree গুলোকে **decorrelated** করে।

গাণিতিক ভিত্তি: B টা স্বাধীন model-এর গড় নিলে variance হয় σ^2/B । কিন্তু correlated হলে variance হয় $\rho\sigma^2 + (1-\rho)\sigma^2/B$ — অর্থাৎ correlation ρ যত কম, variance তত কম। এজন্যই decorrelation-ই আসল জাদু।

OOB (Out-Of-Bag) error: Bootstrap-এ প্রতিটি tree-র জন্য গড়ে ~৩৩% row বাদ পড়ে। ওই বাদ পড়া row গুলো দিয়ে বিনা খরচে validation করা যায়।

Tree বাড়ালে overfit হয়? না। Error একটা সীমায় গিয়ে থেমে যায়। Overfit হয় tree গভীর হলে, সংখ্যা বেশি হলে নয়।

৬. Naive Bayes

Bayes Theorem: $P(C|X) = P(X|C) \cdot P(C) / P(X)$

"Naive" অনুমান: $P(X|C) = P(x_1|C) \times P(x_2|C) \times \dots \times P(x_n|C)$

তাই: $P(C|X) \propto P(C) \times \prod P(x_i|C)$

"Naive" কেন? কারণ ধরে নেওয়া হয় সব feature পরস্পর **conditionally independent** — বাস্তবে প্রায়ই মিথ্যা (Dry Bean-এ Area আর Perimeter তো একসাথেই বাড়ে)। তবু আশ্চর্যজনকভাবে ভালো কাজ করে, কারণ **সঠিক probability**-র চেয়ে **সঠিক ক্রম** (কোন class-এর score বেশি) বেশি জরুরি।

GaussianNB প্রতিটি feature-কে প্রতিটি class-এর ভেতরে normal distribution ধরে, তাই feature গুলো bell-curve আকৃতির হলে ভালো কাজ করে।

Laplace smoothing: কোনো category train-এ না থাকলে $P = 0$ হয়ে পুরো গুণফল 0 করে দেয়। তাই প্রতিটি count-এ 1 যোগ করা হয়।

Metric গুলোর গাণিতিক ভিত্তি

Classification

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

Precision = $TP / (TP + FP)$ · Recall = $TP / (TP + FN)$

F1 = $2 \cdot (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

F1-এ harmonic mean কেন, সাধারণ গড় কেন নয়? Harmonic mean ছোট মানটাকে বেশি গুরুত্ব দেয়। ধরো Precision = 1.0 কিন্তু Recall = 0.0।

সাধারণ গড় = $(1.0 + 0.0) / 2 = 0.5$ → ভুল ধারণা দেয় যে model অর্ধেক ভালো।

Harmonic mean = $2(1 \times 0) / (1 + 0) = 0.0$ → সঠিক, কারণ একটা metric শূন্য হলে model অকেজো।

তাই F1 তখনই বেশি হয় যখন **দুটোই** বেশি।

Precision–Recall Trade-off: threshold কমালে বেশি positive predict হয় → Recall বাড়ে, Precision কমে। বাড়ালে উল্টো। দুটো একসাথে সর্বোচ্চ করা যায় না — কোনটা বেশি দরকার তা সমস্যার উপর নির্ভর করে।

Regression

$R^2 = 1 - SSR/SST$ যেখানে $SSR = \sum (y_i - \hat{y}_i)^2$, $SST = \sum (y_i - \bar{y})^2$

$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$ · $RMSE = \sqrt{MSE}$ · $MAE = (1/n) \sum |y_i - \hat{y}_i|$ · $MAPE = (100/n) \sum |(y_i - \hat{y}_i)/y_i|$

R^2 -এর মানে: SST হলো "শুধু গড় দিয়ে predict করলে কত ভুল হতো", SSR হলো "আমার model-এ কত ভুল হলো"। তাই $R^2 = 0.75$ মানে গড়ের তুলনায় ৭৫% ভুল কমাতে পেরেছি। $R^2 = 0$ মানে গড়ের সমান, **R^2 ঋণাত্মক হলে গড়ের চেয়েও খারাপ** (হ্যাঁ, সম্ভব!)।

RMSE বনাম MAE: RMSE ভুলকে বর্গ করে, তাই বড় ভুল বেশি শাস্তি পায় (outlier-sensitive)। MAE সব ভুলকে সমান দেখে (robust)।

$RMSE \geq MAE$ সবসময় — এদের পার্থক্য যত বেশি, ভুলগুলো তত অসম।

Clustering Algorithm-গুলোর তত্ত্ব

K-Means

$$\text{minimize } \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2 \quad (\text{এটাই WCSS / inertia})$$

Lloyd's algorithm: ① random centroid ② প্রতিটি point নিকটতম centroid-এ ③ centroid = নিজের দলের গড় ④ পুনরাবৃত্তি।

দুইটা তাত্ত্বিক সত্য:

- প্রতি iteration-এ objective **কখনো বাড়ে না** → convergence নিশ্চিত।
- কিন্তু পৌঁছায় **local minimum**-এ। সম্পূর্ণ সমাধান NP-hard — n টা point K দলে ভাগের উপায় প্রায় K^n । এজন্যই `n_init=10`।

সীমাবদ্ধতার তাত্ত্বিক কারণ: objective-এ $\|x - \mu\|^2$ মানে দূরত্ব সব দিকে সমান — এটাই **গোলাকার (spherical)** cluster ধরে নেওয়ার গাণিতিক রূপ। তাই লম্বাটে বা চাঁদ-আকৃতির cluster-এ ব্যর্থ।

Hierarchical Clustering

Agglomerative (bottom-up): প্রতিটি point আলাদা cluster → বারবার নিকটতম দুটো জোড়া → শেষে একটাই।

Linkage	দুই cluster-এর দূরত্ব	চরিত্র
Single	নিকটতম জোড়া (min)	শিকলের মতো লম্বা cluster (chaining)
Complete	দূরতম জোড়া (max)	compact, গোলগাল
Average	সব জোড়ার গড়	মাঝামাঝি
Ward	within-cluster variance-এর বৃদ্ধি সর্বনিম্ন	K-Means-এর সবচেয়ে কাছের

Dendrogram পড়ার নিয়ম: similarity বোঝায় **উল্লম্ব অক্ষ** — কোন উচ্চতায় দুটো branch জোড়া লাগলো। **আনুভূমিক দূরত্বের কোনো মানে নেই**; leaf-এর বাম-ডান সাজানো অনেকটাই যথেষ্ট।

GMM (Gaussian Mixture Model) ও EM

$$p(x) = \sum_{k=1}^K \pi_k \cdot N(x | \mu_k, \Sigma_k) \quad \text{শর্ত: } \sum \pi_k = 1$$

প্রতিটি cluster একটা Gaussian; তিনটা parameter: π (ওজন), μ (কেন্দ্র), Σ (আকার ও ঢাল)।

EM Algorithm — কেন লাগে? কোন point কোন component থেকে এসেছে তা **latent (লুকানো)** variable। জানলে parameter বের করা সোজা, parameter জানলে responsibility বের করা সোজা — কিন্তু কোনোটিই জানা নেই। তাই পালা করে:

E-step: $\gamma_{ik} = \pi_k N(x_i | \mu_k, \Sigma_k) / \sum_j \pi_j N(x_i | \mu_j, \Sigma_j)$ (responsibility)

M-step: γ -কে weight ধরে μ, Σ, π আবার হিসাব

Log-likelihood প্রতি iteration-এ **কখনো কমে না**, তবে local optimum-এ আটকাতে পারে।

K-Means হলো GMM-এর special case: সব covariance সমান ও spherical ($\sigma^2 I$) ধরে নিলে এবং probability-কে hard assignment-এ নামিয়ে আনলে GMM কার্যত K-Means হয়ে যায়। তাই GMM বেশি সাধারণ (general), K-Means তার সরল রূপ।

DBSCAN

তিন ধরনের point:

- **Core point** — eps ব্যাসার্ধের ভেতরে কমপক্ষে min_samples টা প্রতিবেশী আছে
- **Border point** — নিজে core নয়, কিন্তু কোনো core point-এর প্রতিবেশী
- **Noise point** — কোনোটাই নয় $\rightarrow$ label = -1

Cluster তৈরি হয় **density-connected** point-দের নিয়ে — অর্থাৎ core point থেকে শুরু করে ঘনত্বের শিকল ধরে যতদূর যাওয়া যায়।

কেন অদ্ভুত আকৃতি ধরতে পারে? কারণ সে centroid থেকে দূরত্ব মাপে না, বরং **প্রতিবেশীর ঘনত্ব** দেখে ছড়ায়। তাই চাঁদ বা রিং আকৃতির শিকল অনুসরণ করতে পারে।

সীমাবদ্ধতা: ① **ভিন্ন ঘনত্বের** cluster থাকলে একটাই eps সবার জন্য ঠিক হয় না ② উঁচু dimension-এ ঘনত্বের ধারণা ভেঙে পড়ে (curse of dimensionality) ③ eps ও min_samples-এ খুব সংবেদনশীল।

Clustering Metric-এর সূত্র

Silhouette: $s(i) = (b(i) - a(i)) / \max\{a(i), b(i)\}$

$a(i)$ = নিজের cluster-এর অন্যদের সাথে গড় দূরত্ব

$b(i)$ = সবচেয়ে কাছের অন্য cluster-এর সাথে গড় দূরত্ব

$s \approx +1 \rightarrow$ নিজের দলে ভালোভাবে আছে। $s \approx 0 \rightarrow$ সীমান্তে। $s < 0 \rightarrow$ **ভুল cluster-এ পড়েছে।**

একটাই cluster থাকলে $b(i)$ সংজ্ঞায়িত নয়, তাই silhouette হিসাব করা যায় না — কোডে তাই 0 বসানো।

ARI (Adjusted Rand Index): $ARI = (RI - E[RI]) / (\max(RI) - E[RI])$

Rand Index দেখে কতগুলো জোড়া (pair) দুই ভাগে একইভাবে সাজানো হয়েছে। **"Adjusted"** মানে random ভাগ করলেও যতটা মিল হতো, সেটা বাদ দেওয়া — তাই random-এর $ARI \approx 0$, perfect-এর $ARI = 1$

ARI কেন accuracy নয়? Clustering-এর দেওয়া নম্বর যথেষ্ট — cluster "0" মানে class 0 নয়। Accuracy নাম মেলায় বলে ভুল করবে। ARI নাম দেখে না, দেখে **কারা কারা একসাথে আছে** — তাই label permutation-এ অপরিবর্তিত থাকে।

Elbow Method-এর তত্ত্ব

WCSS একটি **monotonically decreasing** function — K বাড়লে সবসময় কমে ($K = n$ হলে 0)। তাই minimize করে K বাছা অর্থহীন। বদলে খুঁজি **diminishing returns**-এর বিন্দু: যেখানে একটা বাড়তি cluster যোগ করে আর তেমন লাভ হচ্ছে না।

কোডে সেটা বের করা হয় জ্যামিতিকভাবে — প্রথম ও শেষ বিন্দু জোড়া রেখা থেকে **সর্বোচ্চ লম্ব দূরত্বের** বিন্দু:

$$d = |(y_2 - y_1)x_0 - (x_2 - x_1)y_0 + x_2y_1 - y_2x_1| / \sqrt{(y_2 - y_1)^2 + (x_2 - x_1)^2}$$

শেষ পাতা — এক নজরে সব সূত্র

বিষয়	সূত্র
Standardization	$z = (x - \mu)/\sigma$
Bias-Variance	$\text{Error} = \text{Bias}^2 + \text{Variance} + \sigma^2$
PCA	$C_v = \lambda_v \cdot \text{PVE} = \lambda_k / \sum \lambda \cdot \sum \phi^2 = 1$
Logistic	$p = 1/(1+e^{-z}) \cdot \log(p/(1-p)) = \beta_0 + \beta_1 x$
SVM	$\text{maximize } 2/\ w\ \cdot \text{RBF: } \exp(-\gamma\ a-b\ ^2)$
Gini	$1 - \sum p_i^2$
Naive Bayes	$P(C X) \propto P(C) \cdot \prod P(x_i C)$
F1	$2PR/(P+R)$
R ²	$1 - \text{SSR}/\text{SST}$
K-Means	$\text{minimize } \sum \sum \ x - \mu_k\ ^2$
GMM	$p(x) = \sum \pi_k N(x \mu_k, \Sigma_k)$
Silhouette	$(b - a)/\max(a, b)$
Elbow	বিন্দু থেকে সরলরেখার লম্ব দূরত্ব